Nama : Ammar
Nim : 2024573010129
Kelas : TI3B
Matkul : Pemograman Mobile (With Flutter)

Tugas: Review Jenis Database dan Supabase dengan Flutter

1. Review Ringkas: Jenis-Jenis Database

Database adalah tempat menyimpan data secara terstruktur agar mudah dicari, diubah, dan dijaga keamanannya. Aplikasi mobile yang datanya perlu disimpan dan dibagikan ke banyak pengguna membutuhkan database di sisi server. Berikut jenis-jenis database yang umum dikenal beserta contoh produknya.

A. Database Relasional (SQL)

- **Deskripsi:** Menyimpan data dalam tabel yang terdiri dari baris dan kolom dengan struktur (skema) yang tetap. Antar tabel dihubungkan oleh kunci (primary key dan foreign key), dan datanya diakses memakai bahasa SQL (SELECT, INSERT, UPDATE, DELETE). Mendukung transaksi ACID sehingga data tetap konsisten.
- **Contoh Produk:** PostgreSQL, MySQL, MariaDB, SQLite (database ringan di dalam perangkat), Microsoft SQL Server, dan Oracle Database.
- **Catatan Praktis:** Cocok untuk data yang terstruktur dan saling berhubungan, seperti menu, pesanan, dan pengguna. Inilah jenis yang dipakai pada praktikum, karena Supabase berjalan di atas PostgreSQL.

B. Database NoSQL

- **Deskripsi:** Istilah untuk database non-relasional yang tidak terikat pada tabel dan skema kaku, sehingga lebih fleksibel dan mudah diskalakan ke banyak server. NoSQL terbagi menjadi beberapa model, yaitu document, key-value, wide-column, dan graph.
- **Contoh Produk:** Document: MongoDB dan Cloud Firestore (Firebase). Key-value: Redis dan DynamoDB. Wide-column: Apache Cassandra. Graph: Neo4j.
- **Catatan Praktis:** Cocok untuk data yang bentuknya berubah-ubah, data berukuran besar, atau akses yang butuh sangat cepat (misalnya cache dan sesi). Kekurangannya, relasi antar data dan query kompleks tidak semudah di SQL. Modul menyebut Firebase sebagai alternatif Supabase yang berbasis NoSQL.

C. Database NewSQL

- **Deskripsi:** Gabungan keunggulan keduanya: tetap memakai model relasional dan SQL beserta transaksi ACID, tetapi dirancang terdistribusi sehingga bisa diskalakan horizontal seperti NoSQL.
- **Contoh Produk:** CockroachDB, TiDB, Google Cloud Spanner, dan YugabyteDB.
- **Catatan Praktis:** Dipakai oleh sistem berskala besar yang butuh konsistensi data ketat sekaligus kapasitas besar, misalnya layanan keuangan. Untuk aplikasi kecil, database SQL biasa sudah cukup.

D. Database Vektor

- **Deskripsi:** Menyimpan data dalam bentuk vektor (embedding), yaitu deretan angka yang mewakili makna teks, gambar, atau suara. Pencariannya berdasarkan kemiripan (similarity search), bukan kecocokan persis.
- **Contoh Produk:** Pinecone, Milvus, Qdrant, Weaviate, Chroma, dan pgvector (ekstensi untuk PostgreSQL).
- **Catatan Praktis:** Banyak dipakai pada fitur AI, seperti pencarian semantik, rekomendasi, dan chatbot yang menjawab dari dokumen. Karena pgvector berjalan di PostgreSQL, fitur ini bisa dipakai di layanan berbasis Postgres. Poin ini berasal dari pengetahuan umum, bukan dari modul.

E. Database Lainnya (Time-Series dan In-Memory)

- **Deskripsi:** Database khusus untuk kebutuhan tertentu. Time-series mengoptimalkan data yang berurutan menurut waktu, sedangkan in-memory menyimpan data di RAM agar aksesnya sangat cepat.
- **Contoh Produk:** Time-series: InfluxDB dan TimescaleDB. In-memory: Redis dan Memcached.
- **Catatan Praktis:** Time-series cocok untuk data sensor dan monitoring, sedangkan in-memory cocok sebagai cache untuk mempercepat aplikasi.

F. Menentukan Pilihan

- Data terstruktur dan saling berhubungan, serta butuh konsistensi: pilih SQL.
- Bentuk data fleksibel atau sangat besar: pertimbangkan NoSQL.
- Butuh SQL sekaligus skala sangat besar: pertimbangkan NewSQL.
- Butuh pencarian berdasarkan makna untuk fitur AI: gunakan database vektor.
- Dalam praktik, satu aplikasi sering memakai lebih dari satu jenis database sekaligus.

2. Review Ringkas: Hubungan Supabase dan Flutter

A. Apa Itu Supabase

- **Fungsi Utama:** layanan Backend as a Service (BaaS), yaitu penyedia backend siap pakai sehingga kita tidak perlu membangun dan mengelola server sendiri.
- **Peran dalam Aplikasi:**
    - **Database sungguhan:** di bawah Supabase terdapat PostgreSQL, yaitu database relasional, bukan NoSQL.
    - **Data API:** tabel dapat diakses lewat HTTP tanpa menulis kode backend. Aplikasi berbicara ke Data API (PostgREST), bukan langsung ke database.
    - **Dashboard:** Table Editor untuk melihat dan menyunting data, SQL Editor untuk menjalankan perintah SQL, dan menu Connect untuk URL serta kunci akses.
- **Catatan Praktis:** Supabase dipilih karena memakai SQL standar yang sudah dikenal dari mata kuliah basis data, gratis untuk skala praktikum, dan memiliki SDK Flutter resmi. Alternatifnya Firebase (NoSQL), Appwrite, atau backend sendiri seperti Laravel atau Express dengan MySQL.

B. Cara Flutter Terhubung ke Supabase

- **Fungsi Utama:** package supabase_flutter menjadi jembatan antara aplikasi Dart dan database Supabase.
- **Peran dalam Aplikasi:**
    - **Instalasi:** package ditambahkan dengan flutter pub add supabase_flutter.
    - **Inisialisasi:** Supabase.initialize() dengan url dan publishableKey dipanggil di main() sebelum runApp(), disertai WidgetsFlutterBinding.ensureInitialized() karena ada await. Klien dipakai lewat Supabase.instance.client.
    - **Operasi CRUD berupa method Dart:** mirip SQL tetapi dalam bentuk method, yaitu select() untuk SELECT, insert() untuk INSERT, update() untuk UPDATE, dan delete() untuk DELETE, dengan filter seperti eq(), gte(), ilike(), order(), dan limit().
    - **Realtime:** mengganti Future dengan Stream melalui stream(primaryKey: [...]), sehingga perubahan data di satu perangkat langsung muncul di perangkat lain.
- **Catatan Praktis:**
    - update() dan delete() wajib diikuti filter seperti eq(). Tanpa filter, seluruh isi tabel akan berubah atau terhapus.
    - Stream dan Future tidak boleh dibuat di dalam build(). Simpan sebagai variabel di StatefulWidget agar koneksi tidak dibuka-tutup terus-menerus.

- Realtime harus diaktifkan untuk tabel di dashboard. Jika belum, stream hanya mengirim data awal tanpa pesan error.
- Bungkus operasi database dengan try/catch dan tangkap PostgrestException agar aplikasi tidak berhenti saat gagal atau offline.

C. Keamanan: Kunci dan Row Level Security (RLS)

- **Fungsi Utama:** mengatur siapa yang boleh membaca dan mengubah data, karena kunci aplikasi berada di dalam aplikasi client dan bisa disalin orang lain.
- **Peran dalam Aplikasi:**
  - **Publishable key** (dulu anon key) aman disertakan di aplikasi karena hanya berlaku sejauh yang diizinkan RLS.
  - **Secret key** (dulu service_role key) menembus RLS dan tidak boleh ada di aplikasi, begitu pula password Postgres. Jika kunci secret pernah bocor atau terunggah ke GitHub, kunci harus segera diganti (rotate).
  - **RLS dan policy:** pengaman di dalam database, bukan di dalam aplikasi. Setelah RLS aktif, tidak ada baris yang bisa diakses kecuali ada policy yang mengizinkan. Peran anon untuk pengunjung yang belum login dan authenticated untuk pengguna yang sudah login.
- **Catatan Praktis:**
  - Policy yang terbuka untuk semua orang (using true) hanya untuk latihan dan prototipe, karena aplikasi belum punya sistem login. Pada pertemuan berikutnya policy dipersempit berbasis pengguna, misalnya auth.uid() = user_id.
  - Pada tabel ber-RLS, delete() hanya menghapus baris yang terlihat oleh policy SELECT. Jika policy SELECT belum ada, delete() tampak berhasil tetapi tidak menghapus apa pun, dan tidak ada pesan error.

Kesimpulan

Database terbagi menjadi beberapa jenis (SQL, NoSQL, NewSQL, vektor, dan lainnya) yang dipilih sesuai kebutuhan data. Supabase memberi Flutter database PostgreSQL yang bisa diakses lewat package supabase_flutter dengan method yang mirip SQL, sehingga aplikasi bisa membaca, menambah, mengubah, dan menghapus data. Tiga hal yang selalu diingat: RLS adalah pengaman di dalam database, update dan delete wajib memakai filter, dan Future maupun Stream tidak boleh dibuat di dalam build().